P3780R0
Detecting bitwise trivially relocatable types

Published Proposal, 2025-06-30

Issue Tracking:
Inline In Spec

Author:
Giuseppe D'Angelo

Audience:
EWG, LEWG

Project:
ISO/IEC 14882 Programming Languages — C++, ISO/IEC JTC1/SC22/WG21

Abstract

We propose a new type trait that detects whether a type can be trivially relocated, and such trivial relocation does not alter any part of the object representation. This trait has direct use cases when creating types that use byte buffers to manage objects, such as certain implementations of `optional`, `inplace_vector`, `any`, `function` and similar.

Table of Contents

§ 1. Changelog

- R0
  - First submission

§ 2. Motivation and scope

[P2786R13] ("Trivial Relocatability For C++26") has been approved for inclusion in C++26. The paper added several new language and library facilities to create, detect and manipulate trivially relocatable types.

A specific feature of [P2786R13]'s design is the **ability of trivially relocate polymorphic types.**

In theory, a polymorphic type is just a type with a "hidden pointer"; from a certain point of view, it's as if objects of polymorphic type contain a non-static data member of pointer type, managed by the implementation, that points to a given's object virtual table. (They may contain more than one such pointer in case of multiple inheritance, but this does not change the discussion in any way.)

Since pointer types are trivially relocatable, it seems natural to think that polymorphic types should be trivially relocatable as well.

However pointers to virtual tables add a complication. Simplifying, on some architectures/ABIs (such as `arm64e`) these pointers are *authenticated*, meaning that some bits of the pointer value are used to store a cryptographic signature. This signature gets verified when one accesses the virtual table pointer (for instance by calling a virtual function).

One of the inputs of the signature is target address of the pointer (that is, the address of the virtual table). Another input, which is crucial to this story, is the *address of the virtual table pointer itself*.

This means that, on those architectures, relocating a polymorphic object by using a simple byte copy would produce an object whose virtual table pointer would fail to authenticate. This is because the new virtual table pointer is now located at a different address than the old one (as it has been moved in memory through relocation), and since the

address is part of the signature, and the signature has been copied bitwise, the signature is now invalid for the new address.

[P2786R13] solves this problem (and allows "universal" trivial relocation of polymorphic types) because trivial relocation happens through a new library entry point: `std::trivially_relocate`. This entry point is strongly typed (unlike `memcpy` or `memmove`). On the architectures that feature this form of pointer signature the implementation of `std::trivially_relocate` is going to take care of re-signing the virtual table pointers as part of the relocation process.

For more information about these mechanisms, please refer to the excellent write-up in P3751R0; to chapter §15.1 of [P2786R13]; and to Apple's documentation of pointer authentication available here.

> ISSUE 1 add a link to the whitepaper on pointer auth once it's available. ¶

Summarizing: in C++26 polymorphic types can be trivially relocatable, and this quality is not platform or implementation-specific.

§ 2.1. The need of detecting bitwise trivial relocations

In some contexts it is necessary to detect whether trivial relocation is akin to a mere byte copy of an object's representation, or if instead `std::trivially_relocate` needs to do some extra work (such as the just discussed re-signing of pointers to virtual tables).

All these context have one in common the fact that one has a wrapper object that manages an array of bytes. Those bytes are then used as storage for dynamically created objects. Do note that such a design is extremely common: it can be used to implement (non-constexpr friendly) `optional`, `inplace_vector`, `any` (with a small object optimization), and many other vocabulary types. The wrapper class may be a class template that knows the type of the object(s) it will hold (`optional`, `inplace_vector`) or be even type-erased (`any`); this does not affect our discussion.

The layout of these wrapper classes looks like this:

```
class Wrapper
{
    alignas(A) std::byte storage[N];
    /* ... other data ... */

public:
    Wrapper();
    ~Wrapper();
    Wrapper(Wrapper &&);
    /* other special member functions, rest of the API, etc. */
};
```

Problems arise if we want to make `Wrapper` a trivially relocatable class. First and foremost, we need to add the new `trivially_relocatable_if_eligible` keyword to the class definition:

```
class Wrapper trivially_relocatable_if_eligible
{
    alignas(A) std::byte storage[N];
    /* ... other data ... */

public:
    Wrapper();
    ~Wrapper();
    Wrapper(Wrapper &&);
    /* other special member functions, rest of the API, etc. */
};
```

The keyword is necessary due to the presence of user-defined destructor and move operations. The non-static data members of `Wrapper` are all of trivially relocatable type (storage of bytes, and some other metadata which is usually integers and pointers), and therefore `Wrapper` is eligible for trivial relocation (see [class.prop]/3); with the addition of the keyword, it is trivially relocatable.

We also need some other requirement:

- If `Wrapper` is *not* a class template (e.g. `any`), we may decide to forbid construction from `T` objects that are not of trivially relocatable types, or decide to always heap-allocate them instead of storing them directly in the internal storage (see § 2.1.1 Example: QVariant). Otherwise, one risks trivially relocating a `Wrapper` object when it contains a non-trivially relocatable type, and that verges on UB (see also § 2.1.5 Resolving LWG4283).
- If `Wrapper` is a class template (e.g. `optional`), then we can make it *conditionally* trivially relocatable depending on `T`. For instance, we can make it inherit from a conditionally trivially relocatable helper class (see also §13.2 of [P2786R13]):

  ```
  // trivially relocatable
  template <bool b>
  struct trivially_relocatable_if {};

  // not trivially relocatable
  template <>
  struct trivially_relocatable_if<false> {
    ~trivially_relocatable_if() {} // deliberately not =default!
  };

  template <typename T>
  class Wrapper trivially_relocatable_if_eligible
    : trivially_relocatable_if<std::is_trivially_relocatable_v<T>>
  {
    /* ... */
  };
  ```

Ultimately, either choice (template or non-template) does not affect our discussion.

The point is that the design of all these wrapper datatypes has a problem with C++26's trivial relocation model; specifically, **the fact that the objects held into the internal storage are of trivially relocatable type is a *necessary but not sufficient condition* to make `Wrapper` itself trivially relocatable.**

A polymorphic type can be trivially relocatable, and stored into `Wrapper`'s internal buffer. However, attempting to trivially relocate a `Wrapper` object that contains such an object of polymoprhic type on an architecture that employs pointer authentication will ultimately result in a runtime crash (UB), because the virtual table pointer in the wrapped object would *not* be re-signed.

We therefore need a more stringent requirement for a type to be stored into `Wrapper`'s internal buffer.

One strategy would be to exclude polymorphic types from being stored into the small buffer (that is, to effectively treat polymorphic types as if they were non-trivially relocatable types).

This however is limiting and possibly confusing, for at least two reasons. First, on platforms where polymorphic types which do not employ any pointer authentication, there is no need of having this additional requirement on the types storable in the small buffer. Second, the check for "is polymorphic" does not immediately convey the nature of the problem.

For these reasons we are proposing a novel trait, that we are going to call `is_bitwise_trivially_relocatable`. This trait is going to be satisfied by types that are trivially relocatable, and also for which trivial relocation is "bitwise"; that is, for which trivial relocation can be achieved without any changes to an object's representation.

For types that are both polymorphic and trivially relocatable, the value of this trait is going to depend on the implementation. This is already the case with `union`s containing polymorphic types (see [class.prop/3]), so the idea is not novel in the core language.

§ 2.1.1. Example: `QVariant`

The `QVariant` class in Qt (documentation) is a type-erased single object wrapper, very similar to the `std::any` class from the Standard Library.

There are a few aspects of `QVariant`'s design that are important to discuss:

1. `QVariant` is *unconditionally trivially relocatable*. In Qt this is realized via the Qt's specific trivial relocation macros, but of course in C++26 this would be achieved by using the new language keywords.
2. `QVariant` features a small object optimization, just like the `Wrapper`example discussed before. When an object is stored into a `QVariant`, if the object is small enough, not overaligned, etc. *and* it's of trivially relocatable type, then it gets stored directly into `QVariant`'s small buffer. Otherwise, the object is stored in heap-allocated storage, and the `QVariant` object stores a pointer to it.

In other words, QVariant class layout looks like this (please note, this is a huge simplification; the actual code is available here):

```
class QVariant trivially_relocatable_if_eligible
{
    // Storage: pointer to heap-allocated object, or small buffer
    union Storage {
      void *m_ptr = nullptr;
      alignas(QVariantSmallBufferAlignment) std::byte m_buffer[QVariantSmallBufferSize];
    } m_storage;

    // Pointer to a data structure with pointers to functions to do
    // type-aware manipulation of the storage (copy, move, destroy,
    // assign, etc.), answer type queries, and so on:
    const detail::type_interface *m_type_interface = nullptr;

public:
    QVariant();
    ~QVariant();
    QVariant(const QVariant &);
    QVariant(QVariant &&);

    // ... other special member functions, constructors from a generic
    // T, rest of the API, etc.; not important for the discussion.
};

static_assert(std::is_trivially_relocatable_v<QVariant>); // OK
```

Again, the requirement that the wrapped type needs to be trivially relocatable in order to be stored into the small storage is insufficient: `T` needs to be *bitwise* trivially relocatable.

§ 2.1.2. Example: `optional`

Consider this (non-constexpr friendly) implementation of `optional`, from this blog post by Arthur O'Dwyer, with some code omitted for brevity:

```
template<class T>
class [[trivially_relocatable(std::is_trivially_relocatable_v<T>)]] Optional {
public:
  explicit Optional() {}

  template<class... Args>
  void emplace(Args&&... args) {
    if (engaged_) {
      std::destroy_at(t());
      engaged_ = false;
    }
```

```
    std::construct_at(t(), std::forward<Args>(args)...);
    engaged_ = true;
  }

  T& value() { assert(engaged_); return *reinterpret_cast<T*>(data_); }
  const T& value() const { assert(engaged_); return *reinterpret_cast<const T*>(data_);

  // ... rest of the API ...

private:
  T *t() { return reinterpret_cast<T*>(data_); }

  alignas(T) char data_[sizeof(T)];
  bool engaged_ = false;
};
```

Apart from using [P1144R13] trivial relocation syntax, this `Optional` type is yet another instance of `Wrapper`. An `Optional` wrapping a trivially relocatable polymorphic type may misbehave if it gets trivially relocated.

2.1.3. Example: `inplace_vector`

A non-constexpr friendly `inplace_vector` is just a generalization of the just shown `Optional`, holding storage for up to N elements rather than just one. A quick modification of the previous example yields:

```
template <typename T, std::size_t N>
class inplace_vector
    : trivially_relocatable_if<std::is_trivially_relocatable_v<T>>
{
public:
  explicit inplace_vector() = default;

  T &push_back(const T &value)
  {
      if (m_size == capacity()) throw std::bad_alloc();
      std::construct_at(t()[m_size], value);
      return t()[m_size++];
  }

  // ... rest of the API ...

private:
  T *t() { return reinterpret_cast<T *>(m_data); }

  alignas(T) std::byte m_data[sizeof(T) * N];
  size_t m_size = 0;
};
```

The very same issue is present here.

§ 2.1.4. Example: `QList`

In Qt 4 and 5 `QList<T>` managed a dynamic array of "blocks", with each block being a few bytes. (In Qt 6 the implementation is completely different.)

For a type `T` which was small enough, not overaligned, and trivially relocatable, objects of that type were stored *directly* into each block (possibly with extra padding). Objects of types that didn't meet these criteria were individually allocated on the heap, and the blocks stored pointers to them.

With a few stylistic changes, the code looked like this:

```
template <typename T>
class QList
{
public:
    // Tag type, used for tag dispatching depending on the actual layout
    struct MemoryLayout
        : std::conditional<
            !QTypeInfo<T>::isTriviallyRelocatable || QTypeInfo<T>::isLarge,
            QListData::IndirectLayout,
            typename std::conditional<
                sizeof(T) == sizeof(void*),
                QListData::ArrayCompatibleLayout,
                QListData::InlineWithPaddingLayout
             >::type>::type {};
    /* ... */
};
```

(Source.)

The main idea behind this implementation strategy has to do with type erasure; specifically, **the management of the dynamic array backing a `QList<T>` object was type erased**. Allocation and reallocation of the array happened through calls to `realloc`; inserting and erasing in the middle would "open a window" and "close a window" by calling `memmove` on the tail of the array, that is, shifting the tail's elements around.

In other words, `QList<T>` was a random access container, but traded speed of access and overall memory usage (compared to something like a `std::vector<T>`) for a much smaller generated code size and better compilation times.

The implementation of `QList` had the same issue of `Wrapper`: **a `QList<Polymorphic>` must not store objects of type `Polymorphic` directly inside each block unless they're *bitwise* trivially relocatable.**

2.1.5. Resolving LWG4283

The introduction of the trait affects the resolution of [LWG4283].

This issue argues that the specification of `std::trivially_relocate` in [obj.lifetime] is incomplete: there is no precondition that covers the fact that trivially-relocatable objects like `Wrapper` must be providing storage to objects of **bitwise** trivially relocatable types, otherwise the behavior is undefined.

With the trait that we are proposing, we would change the proposed resolution of the issue to something much more simple and direct, like this:

> *Preconditions:*
>
> - [...]
> - (10.4) For each object `o` whose storage is being provided for ([intro.object]) by objects in the `[first, last)` range, let `O` be the type of `o`; `is_bitwise_trivially_relocatable_v<O>` is `true`.

There is no need for the second change that explicitly mentions polymorphic types, nor to mention `union`s at all (as proposed on the Library reflector.)

2.2. Aside: is trivial relocation... trivial?

Historically trivial relocation has always been implemented in third-party libraries in terms of byte operations.

Lacking support from the core language, libraries introduced ways to mark types as "trivially relocatable" (macros, template variables to specialize, etc.), and exploited "UB that works in practice" by using `memcpy`/`memmove` to move in memory trivially relocatable objects, `realloc` to reallocate contiguous buffers of trivially relocatable objects, and so on. [P1144R11] and [P3559R0] contain extensive surveys of such libraries.

Setting aside the fact that all such library solutions are formally UB, the evolution of C++ proposals for trivial relocation "diverged" from the status quo in at least two aspects:

1. the semantic split of "trivial relocation" and "replacement". Most libraries (see [P3559R0]) combine both semantics under one trait. As highlighted by [P3233R0], this semantic combination is necessary to use trivial relocation to optimize not just vector reallocation, but also vector erase, insertion, swaps in algorithms, and so on (see also [P3278R0]). It is fair to argue that the two type properties are, indeed, independent properties, and therefore should be modeled separately. This design got incorporated in [P2786R7] revision 7.
2. The fact that trivial relocation may not be equivalent to a byte copy. This is a much recent development, driven by the introduction of pointer authentication on new platforms, while preserving the ability of polymorphic types to be trivially relocatable.

One may argue that C++26's trivial relocation is not "truly" trivial, because in general it is not equivalent to a byte copy. On the other hand, one may counter-argue that it's trivial in the sense that no user-defined code is executed. We don't think this particular debate is productive.

What however is really important is the upgrade story for all these libraries: how can they switch from their own custom traits and algorithms towards standardized solutions?

Here we see a potential conflict: many libraries today define "trivial relocatable" as a combination of *bitwise trivially relocatable* and *replaceable*. C++26's trivially relocatable type traits do not give exactly these semantics, and this may impede adoption of the new facilities; or, worse, it may introduce regressions in code that was *expecting* bitwise trivial relocatability, but gets ported to just check for trivial relocatability.

We can also foresee that some libraries (e.g. Qt) will continue to use "UB that works in practice" because of deficiencies of C++26's APIs. For instance, it is reasonable to expect that Qt will keep using `realloc` to e.g. grow a buffer containing bitwise trivially relocatable objects, as that provides a huge benefit over allocating a new buffer, trivially relocating in there, and deallocating the old buffer. However, C++26 is lacking a trivially relocation-aware version of `realloc`, nor it granted `realloc` (or similar APIs) the ability to automatically bitwise trivially relocate objects in the memory being reallocated.

In conclusion, adding a type trait that closely models what these libraries have been assuming in the past (20+ years, in the case of Qt) will likely provide a better upgrade path for them.

2.2.1. Polls during the [P1144R13] review in Sofia

During the LEWG review in Sofia of [P1144R13] a good part of the discussion focused on the fact that C++26's trivial relocatability does not necessarily imply bitwise trivial relocability.

Some raised the idea of a separate trait; this poll was taken:

> **POLL:** We encourage more exploration on a trait and/or a function equivalent to "bitwise_trivially_relocate" that behaves in a similar way to what's described in "3.13 [obj.lifetime]" section on P1144R13 (enable library utils to realloc/memcpy).
>
> ```
> SF | F | N | A | SA
> 6  | 7 | 4 | 6 | 2
> ```
>
> Outcome: Feeling of the room poll. Some want more exploration (very weakly)

While this paper proposes such a trait, please do note that we are *not* proposing to allow `memcpy`, `realloc` etc. to perform trivial relocation.

3. Design decisions

3.1. Do we need a new trait? Can users implement this feature?

In principle, users can directly detect whether they're on a platform where trivial relocation may not result in a byte copy, for instance by checking platform-specific macros. At the time of this writing, to the best of our knowledge, `arm64e` is the only ABI that features such a pointer authentication scheme; users could therefore check if `__arm64e__` is defined.

An in-language portable solution is also possible. As discussed in [P2786R13] in chapter §15.1, `union`s containing polymorphic types may or may not be trivially relocatable.

The problem with unions and polymorphic types is exactly the same as the one illustrated in § 2.1 The need of detecting bitwise trivial relocations. Given an union like this one:

```
union U
{
    int m_int;
    Polymorphic m_poly;
};
```

the implementation cannot know which member is active. Therefore, when trivially relocating an object of type `U`, should the implementation just copy the byte representation (if the `m_int` member is active) or copy the byte representation **and also** re-sign the virtual table pointer (if the `m_poly` member is active)?

Note that "guessing wrong" breaks the program, so that is not a possibility. On the other hand, if the target platform does not have pointer authentication, there there is no issue with trivially relocating the union, as we can just do a byte copy.

As already mentioned, this issue got enshrined in [class.prop/3]: it is implementation-defined whether the `U union` is trivially relocatable or not.

We can use this knowledge to build a detector:

```
template <class T>
union Detector
{
    int dummy;
    T t;
};

template <class T>
constexpr bool is_bitwise_trivially_relocatable_v
    = std::is_trivially_relocatable_v<Detector<T>>;
```

In conclusion, building the proposed facilities is indeed theoretically possible in user code.

In practice, **we reject the idea that users should reinvent this "gadget" in any library that provides type-erasure facilities.** That is to say, **we firmly believe that such detection has to be built in the Standard Library.** Relying on the fact that a polymorphic type is bitwise trivially relocatable if and only if a union containing a member of that type is trivially relocatable requires some extremely specialized knowledge.

§ 3.2. Why a new type trait, rather than a non-template boolean variable / a macro / etc.?

An alternative design could be to simply have a predefined macro or a *non-template* `constexpr bool` variable that reflects the platform-specific property of "is relocation bitwise for polymorphic types".

Although they would achieve the same goal, we believe that a new type trait is much more expressive than combining checks.

For instance, in order to check whether a type `T` can be stored into `QVariant`'s small buffer implementation (see § 2.1.1 Example: QVariant), compare the proposed new trait:

```
template <typename T>
constexpr bool CanBeStoredInQVariantSmallBuffer =
  std::is_bitwise_trivially_relocatable_v<T> &&
  /* check alignment, size, ... */;
```

with the alternative:

```
template <typename T>
constexpr bool CanBeStoredInQVariantSmallBuffer =
  std::is_trivially_relocatable_v<T> &&
  (!std::is_polymorphic_v<T> || std::trivial_relocation_is_bitwise) &&
  /* check alignment, size, ... */;
```

The proposed trait makes it much more clear what is the requirement on `T` that we are looking for, without the need to deploying a material implication that we are in one of the problematic cases (polymorphic types). The second form could use the implication operator proposed by [P2971R3] for some more clarity, but we still strongly prefer the first form.

§ 4. Impact on the Standard

This proposal is a pure library addition; it adds a new trait (class template and associated variable template) to the `<type_traits>` header.

§ 5. Proposed Wording

All the proposed changes are relative to [N5008].

§ 5.1. Feature-testing macro

In [version.syn], bump the value of the `__cpp_lib_trivially_relocatable` macro to match the date of adoption of the present proposal:

```
#define __cpp_lib_trivially_relocatable            ~~202502L~~??????L
  // freestanding, also in <memory>, <type_traits>
```

§ 5.2. Wording

In [meta.type.synop], amend the `<type_traits>` synopsis as shown:

```
// [meta.unary.prop], type properties
template<class T> struct is_const;
template<class T> struct is_volatile;
template<class T> struct is_trivially_copyable;
template<class T> struct is_trivially_relocatable;
template<class T> struct is_bitwise_trivially_relocatable;
template<class T> struct is_replaceable;
template<class T> struct is_standard_layout;
```

Add the relative variable template definition:

```
template<class T>
  constexpr bool is_trivially_relocatable_v = is_trivially_relocatable<T>::value;
template<class T>
  constexpr bool is_bitwise_trivially_relocatable_v =
    is_bitwise_trivially_relocatable<T>::value;
template<class T>
  constexpr bool is_standard_layout_v = is_standard_layout<T>::value;
```

Add a new row to Table 54 ("Type property predicates") in [meta.unary.prop], just after the row for `is_trivially_relocatable`:

Template	Condition	Preconditions
`template<class T> struct is_bitwise_trivially_relocatable;`	`is_trivially_relocatable<T>` is `true`, and trivially relocating objects of type `T` by means of calling `trivially_relocate` ([obj.lifetime]) does not change the object representation of the objects being relocated. [*Note* *: Depending on the implementation, objects of types that are polymorphic and trivially relocatable may change object representation after being relocated by a call to `trivially_relocate`. — *end note*]	`remove_all_extents_t<T>` shall be a complete type or *cv* `void`.

6. Acknowledgements

Thanks to KDAB for supporting this work.

All remaining errors are ours and ours only.

References

Informative References

[LWG4283]
Giuseppe D'Angelo. *std::trivially_relocate needs stronger preconditions on "nested" objects with dynamic lifetime*. URL: https://cplusplus.github.io/LWG/issue4283

[N5008]
Thomas Köppe. *Working Draft, Programming Languages — C++*. 15 March 2025. URL: https://wg21.link/n5008

[NonConstexprOptionalBlog]
Arthur O'Dwyer. *Non-constexpr Optional and trivial relocation*. URL: https://quuxplusone.github.io/blog/2025/05/09/siren-song-3-optional/

[P1144R11]
Arthur O'Dwyer. *std::is_trivially_relocatable*. 15 May 2024. URL: https://wg21.link/p1144r11

[P1144R13]
Arthur O'Dwyer, Artur Bać, Daniel Liam Anderson, Enrico Mauro, Jody Hagins, Michael Steffens, Stéphane Janel, Vinnie Falco, Walter E. Brown, Will Wray. *std::is_trivially_relocatable*. 15 May 2025. URL: https://wg21.link/p1144r13

[P2786R13]
Pablo Halpern, Joshua Berne, Corentin Jabot, Pablo Halpern, Lori Hughes. *Trivial Relocatability For C++26*. 14 February 2025. URL: https://wg21.link/p2786r13

[P2786R7]
Mungo Gill, Alisdair Meredith, Joshua Berne. *Trivial Relocatability For C++26*. 17 September 2024. URL: https://wg21.link/p2786r7

[P2971R3]
Walter E Brown. *Implication for C++*. 13 January 2025. URL: https://wg21.link/p2971r3

[P3233R0]
Giuseppe D'Angelo. *Issues with P2786 (Trivial Relocatability For C++26)*. 16 April 2024. URL: https://wg21.link/p3233r0

[P3278R0]
Nina Ranns. *Analysis of interaction between relocation, assignment, and swap*. 22 May 2024. URL: https://wg21.link/p3278r0

[P3559R0]
Arthur O'Dwyer. *Trivial relocation: One trait or two?*. 8 January 2025. URL: https://wg21.link/p3559r0

§ Issues Index

ISSUE 1 add a link to the whitepaper on pointer auth once it's available. ↵